

CS3211 Concurrent Programming

AY 25/26 Sem 2 — github/omgeta

1. Introduction

Concurrency refers to a system performing multiple independent tasks in overlapping time periods.

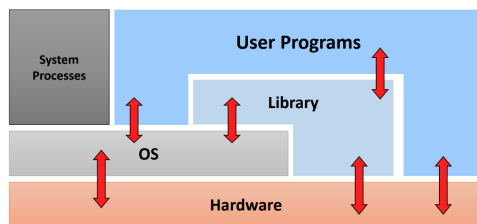
Concurrency vs. Parallelism:

- Concurrency: tasks progress in overlapping time
- Parallelism: tasks also execute simultaneously
 - Enabled by multiple cores, each can have multiple hardware threads (SMT)
 - Hardware threads dictate parallel execution of kernel threads

Program Parallelization Steps:

- Decomposition: split problem into parallel tasks
 - Granularity: size of task
- Scheduling: assign tasks to threads
- Orchestration: synchronisation and communication of tasks to satisfy dependencies
- Mapping: bind threads to physical cores

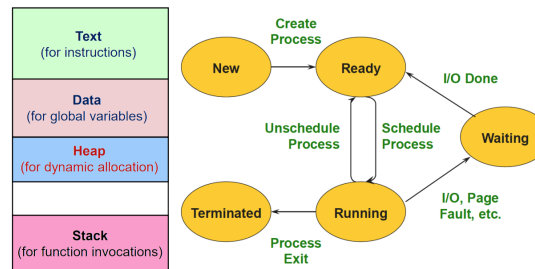
Software Stack:



Processes

Process is an abstraction for a running program:

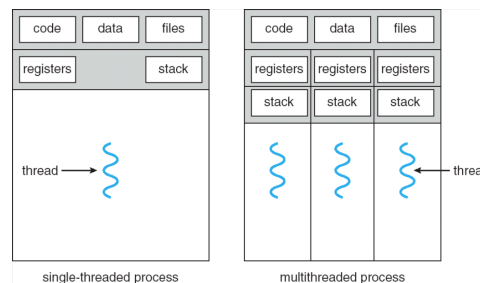
- High system call overhead
- OS allocates and initializes data structures
- Communication costly through the OS



Threads

Threads are independent execution flows within a process:

- Shared: Resources, Address Space
 - Private: Thread ID, Registers, "Stack" (diff. SP)
- + Efficient: cheaper creation and context switching
 - + Resource Sharing: process resources can be shared



Implementations:

- User-Threads are managed by a userspace library:
 - + Fast: context switches have low overhead
 - Cannot parallelize to different cores
 - Threads blocking will block whole process
- Kernel-Threads are implemented in the OS:
 - + Parallel: threads can run across many cores
 - Slower: thread operations are system calls

Synchronisation

Race Condition:

- Program outcome depends on timing/ordering of concurrent operations

Data Races lead to undefined behaviour:

- Two concurrent access to same memory location
- At least one access is a write
- No synchronization

Mutual Exclusion synchronises access to shared resources:

- Thread locks mutex to access shared resources
- Same thread unlocks mutex once shared resources no longer used

Symptoms of Incorrect Synchronization:

- Deadlock: there is an execution where no progress is made when it should be possible
 - Indistinguishable from livelock
- Starvation: a thread cannot progress because other threads hold a required resource, which it does not get a chance to acquire

Mechanisms:

- Lock: primitive `acquire()`, `release()`
- Semaphore: `atomic ≥ 0`; `acquire()`, `release()`
 - Binary: represents single access
 - Counting: number of threads in CS $\leq N$
 - Can be manipulated by different threads
 - No relation to data being controlled
- Monitors: high-level implicit concurrency
- Messages: explicit model of communication and synchronisation based on atomic data transfer

2. C++

Threads

Operations:

- i. Construct:
 - Function: `std::thread t(f, args...)`
 - Callable object: `std::thread t{F{ }, args...}`
 - Lambda: `std::thread t([ ]{ ... })`
- ii. Wait Once: `t.join()`
 - Check: `t.joinable()`
 - Must still join on exception paths
- iii. Detach: `t.detach()`
 - Detached thread no longer joinable
 - May outlive local variable references
- iv. Pass By Reference: `std::ref(x)`
- v. Ownership: movable, not copyable

Mutexes

Operations:

- i. Construct: `std::mutex m`
- ii. Explicit: `m.lock()`, `m.unlock()`
 - Must unlock on every exit path, incl. exceptions
- iii. RAII: `std::lock_guard<std::mutex> g(m)`
 - + Locks on construction and unlocks on scope exit
- iv. Flexible: `std::unique_lock<std::mutex> lk(m)`
 - Adopt Existing Lock: `lk(m, std::adopt_lock)`
 - Defer Lock: `lk(m, std::defer_lock)`
 - + Unlocks on scope exit if mutex owned
 - + Allows multiple lock/unlock within a scope
 - + Needed for condition variables
 - + Moveable: can transfer lock ownership
- v. Multiple Mutexes (Deadlock-Free):
 - Lock in total order, avoid nested locks
 - Explicit: `std::lock(m1, m2, ...)`
 - RAII: `std::scoped_lock lk(m1, m2, ...)`

Condition Variables

Condition variables sync actions without busy-waiting.

Operations:

- i. Construct: `std::condition_variable cv`
 - Works only on `std::unique_lock<std::mutex>`
 - Generic (any mutex-like, less efficient):
`std::condition_variable_any cva`
- ii. Wait: `cv.wait(lk)` / `cv.wait(lk, pred)`
 - Checks predicate with mutex locked
 - If condition is met, returns
 - If condition is unmet, unlocks mutex and blocks
 - Spurious Wake: may wake without notification to check condition; so should be side effect-free
- iii. Wake One Thread: `cv.notify_one()`
- iv. Wake All Threads: `cv.notify_all()`

Monitors

Monitors encapsulate shared data and operations enforced by a mutex and condition variables.

- i. One thread executes monitor operations at a time
- ii. Threads wait for conditions and state changes

Example:

```
class JobQueue {
    std::mutex m;
    std::condition_variable cv;
    std::queue<Job> q;
public:
    void enqueue(Job j) {
        {
            std::unique_lock<std::mutex> lk(m);
            q.push(j);
        }
        cv.notify_one();
    }

    Job dequeue() {
        std::unique_lock<std::mutex> lk(m);
        cv.wait(lk, [&]{ return !q.empty(); });
        Job j = q.front(); q.pop();
        return j;
    }
};
```

Memory Model

Memory model is the contract about the visible side effects of memory operations.

- i. As-if: compiler/hardware may reorder operations if single-threaded observable behavior is unchanged.

Ordering Relations:

- i. Sequenced-Before: program order
- ii. Synchronizes-With: suitably-tagged store $\rightarrow$ load on same atomic, if the load reads that value
- iii. Happens-Before: transitive closure of Sequenced-Before $\cup$ Synchronizes-With
- iv. Modification Order: per-atomic total order of writes consistent across threads
- v. Coherence Order: per-atomic partial order of writes (totally ordered) and reads (unordered w.r.t others) consistent across threads
- vi. Visible Side-Effect: write visible to a read

Atomics (`std::atomic<T>`) have indivisible operations:

- i. Load: `a.load(std::memory_order)`
- ii. Store: `a.store(val, std::memory_order)`
- iii. RMW: `a.fetch_add(val, std::memory_order)`,
`a.exchange(val, std::memory_order)`
- iv. May be lock-free/implemented with mutex:
`a.is_lock_free()`, `T::is_always_lock_free`

Memory Models:

- i. Sequentially Consistent (`seq_cst`):
 - All threads agree on same order of operations
- ii. Relaxed (`relaxed`):
 - No Synchronizes-With cross-thread guarantees
- iii. Acquire-Release (`acquire`, `release`, `acq_rel`)
 - Release store $\rightarrow$ Acquire load on same atomic can establish Synchronizes-With
 - Writes before release become visible after the acquire (in the acquiring thread)
 - Release Sequence: chain of RMWs on the same atomic after release; an acquire reading from the chain synchronizes-with the original release

Lock-based Data Structures

Lock-based structures protect shared state with mutexes.

- Lock the right mutex before accessing protected data
- Hold locks for the minimum time
- Interface should not expose race conditions
- Deadlocks are possible

Stack (one global mutex)

```
void push(T x) {
    std::lock_guard<std::mutex> lk(m);
    data.push(std::move(x));
}

std::shared_ptr<T> pop() {
    std::lock_guard<std::mutex> lk(m);
    if (data.empty()) throw empty_stack();
    auto res =
        std::make_shared<T>(std::move(data.top()));
    data.pop();
    return res;
}
```

- + Safe for concurrent calls, exception-safe
- Work is serialized
- No built-in waiting for items: must poll `empty()` or catch `empty_stack` exceptions

Queue (one mutex + notification)

```
void push(T x) {
    std::lock_guard<std::mutex> lk(mut);
    data_queue.push(std::move(x));
    data_cond.notify_one();
}

void wait_and_pop(T& value) {
    std::unique_lock<std::mutex> lk(mut);
    data_cond.wait(lk, [this]{ return
        !data_queue.empty(); });
    value = std::move(data_queue.front());
    data_queue.pop();
}
```

- + `wait_and_pop()` blocks until an item is available
- + Safe for concurrent member calls
- One mutex still serializes queue operations
- `notify_one()` + exception in `wait_and_pop()` can leave other waiters asleep

Queue (push shared-pointers in the queue)

```
void push(T x) {
    auto data = std::make_shared<T>(std::move(x));
    std::lock_guard<std::mutex> lk(mut);
    data_queue.push(data);
    data_cond.notify_one();
}

std::shared_ptr<T> wait_and_pop() {
    std::unique_lock<std::mutex> lk(mut);
    data_cond.wait(lk, [this]{ return
        !data_queue.empty(); });
    auto res = data_queue.front();
    data_queue.pop();
    return res;
}
```

- + Data can be allocated before locking
- + Returned value can share ownership with the queue element
- One mutex still serializes queue operations

Queue (fine-grained; dummy node)

```
node* get_back() {
    std::lock_guard<std::mutex> lk(back_mutex);
    return back;
}

std::unique_ptr<node> pop_front() {
    std::lock_guard<std::mutex> lk(front_mutex);
    if (front.get() == get_back()) return nullptr;
    auto old_front = std::move(front);
    front = std::move(old_front->next);
    return old_front;
}

void push(T x) {
    auto new_data =
        std::make_shared<T>(std::move(x));
    auto p = std::make_unique<node>();
    node* new_back = p.get();
    std::lock_guard<std::mutex> lk(back_mutex);
    back->data = new_data;
    back->next = std::move(p);
    back = new_back;
}
```

- + Separate `front_mutex` and `back_mutex`
- + `push()` only locks the back
- + More concurrency than one global mutex
- Locking protocol and empty-check become more complex

Queue (more fine-grained)

```
std::unique_ptr<node> try_pop_head() {
    std::unique_lock<std::mutex>
        front_lk(front_mutex);
    if (front.get() == get_back()) return nullptr;

    std::unique_lock<std::mutex>
        node_lk(front->mutex);
    auto old_head = std::move(front);
    front = std::move(old_head->next);
    return old_head;
}

std::shared_ptr<T> try_pop() {
    std::unique_ptr<node> old_head;
    { old_head = try_pop_head(); } // lock
    released before delete
    return old_head ? old_head->data :
        std::shared_ptr<T>();
}

void push(T x) {
    auto new_data =
        std::make_shared<T>(std::move(x));
    auto p = std::make_unique<node>();
    node* new_back = p.get();

    std::lock_guard<std::mutex> back_lk(back_mutex);
    std::lock_guard<std::mutex>
        node_lk(back->mutex);
    back->data = new_data;
    back->next = std::move(p);
    back = new_back;
}
```

- + Per-node mutex
- + More fine-grained synchronization between `push()` and `pop()`
- More complex locking protocol
- Must release locks before destroying the old node

Lock-Free Data Structures

Lock-free structures avoid blocking library calls.

- i. Obstruction-free: a thread completes if it eventually runs in isolation
- ii. Lock-free: from any point, some thread completes
 - Livelock-free but Starvation possible
 - + Maximum concurrency: some thread progresses
 - + Robust: if one thread dies, others can proceed
 - Atomics may be much slower than non-atomics
 - Contention/write propagation hurt performance
 - Cache ping-pong and false sharing
- iii. Wait-free: every running thread completes regardless of execution speed of others
 - Livelock- and Starvation-free; stronger lock-free
 - + Per-thread progress guaranteed
 - Harder to design and prove correct
 - Often higher overhead than lock-free structures

Risks:

- i. Livelock: from unbounded restarts in response to other thread actions
- ii. Starvation: loops with CAS, or algorithms with unbounded number of retries due to clashes

Guidelines:

- i. Start with `std::memory_order_seq_cst`
- ii. Use CAS loops carefully
- iii. Watch for ABA
- iv. Use safe memory reclamation
- v. Prefer lock-based unless lock-free progress needed

CAS Loop Pattern

```
int fetch_add(std::atomic<int>& value, int add) {
    int old_value = value.load();
    while (true) {
        int new_value = old_value + add;
        if (value.compare_exchange_weak(old_value,
                                         new_value))
            return old_value;
    }
}
```

Queue (initial idea)

```
void push(Job job) {
    Node* new_dummy = new Node();
    Node* work_node =
        m_queue_back.exchange(new_dummy);
    work_node->job = job;
    work_node->next.store(new_dummy);
}

std::optional<Job> try_pop() {
    Node* old_front = m_queue_front.load();
    while (true) {
        Node* new_front = old_front->next.load();
        if (new_front == QUEUE_END)
            return std::nullopt;
        if (m_queue_front.CAS(old_front, new_front))
            return old_front->job;
    }
}
```

- `exchange()` gives each producer its exclusive dummy node
- `try_pop()` advances the head with a CAS loop

Issue 1: Data races

```
Node* work_node =
    m_queue_back.exchange(new_dummy,
                          stdmo::acq_rel);
work_node->next.store(new_dummy, stdmo::release);

Node* old_front =
    m_queue_front.load(stdmo::acquire);
Node* new_front =
    old_front->next.load(stdmo::acquire);
m_queue_front.compare_exchange_weak(old_front,
                                     new_front, stdmo::acq_rel);
```

- Producer vs. producer: `m_queue_back.exchange(..., stdmo::acq_rel)`
- Producer vs. its consumer: release store to `next`, acquire load from `next`
- Producer vs. next consumer: acquire / `acq_rel` accesses on `m_queue_front`

Issue 2: ABA

```
struct alignas(16) GenNodePtr {
    Node* node;
    uintptr_t gen;
};

std::atomic<GenNodePtr> m_queue_front;

• CAS may succeed even if a pointer changes away from A and back to A
• Fix: add a generation counter to the front pointer
```

Issue 3: Use-after-free

```
Node* get_recycled_node_or_allocate_new();
void add_node_to_recycling_stack(Node* node);

• Old front pointers may become stale
• Fix: do not delete nodes directly; reuse nodes via a lock-free recycling stack
```

Issue 4: Recycling race

```
Node* cur_stack_next =
    old_stack_top.node->next.load(stdmo::acquire);

node->next.store(old_stack_top.node,
                stdmo::release);

• Recycling also needs synchronization
• Use release / acquire on recycled-node next
```

Issue 5: Serializability

- The repaired queue can still be non-serializable: head / tail may be updated before fully linked
- Fix: Michael-Scott Queue prioritizes maintaining a valid linked list

Classes

Class Skeleton:

```
class C {
    std::string x;
public:
    C(): x("default") {
        // default constructor
    }

    C(int i): x(std::string(i)) {
        // custom constructor
    }

    C(const C& other) : x(other.x) {
        // copy constructor
    }

    C& operator=(const C& other) {
        // copy assignment operator
        x = other.x;
        return *this;
    }

    C(C&& other): x(std::move(other.x)) {
        // move constructor
    }

    C& operator=(C&& other) {
        // move assignment operator
        x = std::move(other.x);
        return *this;
    }

    ~C() {
        // destructor
    }
};
```

Rule of 3: if a class needs a custom destructor, copy constructor, or copy assignment operator then it usually needs all 3

Shared Mutex

Readers-Writers:

```
class ThreadSafeCounter {
    std::shared_mutex mutex_;
    unsigned int value_{0};
public:
    ThreadSafeCounter() = default;

    // Multiple threads/readers can
    // read the counter's value at the same
    // time.
    unsigned int get() const
    {
        std::shared_lock lock(mutex_);
        return value_;
    }

    // Only one thread/writer can
    // increment/write the counter's value.
    void increment()
    {
        std::unique_lock lock(mutex_);
        ++value_;
    }

    // Only one thread/writer can
    // reset/write the counter's value.
    void reset()
    {
        std::unique_lock lock(mutex_);
        value_ = 0;
    }
};
```

Smart Pointers

Smart pointers manage heap resources automatically.

Ownership:

- i. `std::unique_ptr<T>`: exclusive ownership
- ii. `std::shared_ptr<T>`: shared ownership via atomic reference counting
- iii. `std::weak_ptr<T>`: non-owning observer of a `shared_ptr`

Basic Methods:

- i. Construct:
 - `auto p = std::make_unique<T>(args...)`
 - `auto p = std::make_shared<T>(args...)`
- ii. Access:
 - Dereference: `*p`, `p->member`
 - Raw pointer: `p.get()`
- iii. Reset / replace:
 - `p.reset()`
 - `p.reset(new T(...))`
- iv. Move ownership:
 - `std::unique_ptr` is movable, not copyable
 - `auto q = std::move(p)`
- v. Shared count:
 - `p.use_count()`
 - Copying a `shared_ptr` increments the count
- vi. Weak access:
 - `w.lock()` gives a `shared_ptr` if object still exists
 - `w.expired()` checks whether object has already been destroyed
- vii. Unique-pointer release:
 - `p.release()` returns raw pointer and gives up ownership

3. Go

Goroutines

Goroutines are lightweight function tasks running in the same address space.

Operations:

- i. Start: `go f(args...)`
- ii. Lifetime:
 - Program exits when `main` returns
 - Not GC'd: avoid goroutine leaks
- iii. Blocks only block the thread, no other goroutines
- iv. Preemptable: by Go's runtime

Channels

Channels coordinate goroutines by communication.

Operations:

- i. Make: `ch := make(chan T) / make(chan T, C)`
 - Directional:
`chan<- T` (send-only), `<-chan T` (recv-only)
- ii. Send: `ch <- v`
 - Blocks if channel is full (buffered) / until a receiver (unbuffered)
- iii. Receive: `v, ok := <-ch`
 - Blocks if channel is empty (can cause deadlocks)
 - `ok false` $\iff$ `v` is default from closed channel
- iv. Close: `close(ch)`
 - Allowed from closed channel any number of times
 - Sending to a closed channel panics
- v. Range: `for v := range ch { ... }` runs until channel is closed and drained

Ownership Pattern:

- i. Owner goroutine makes, writes, and closes channel
- ii. Expose only `<-chan T` to consumers

select

`select` composes channel operations:

- i. Blocks if no case is ready (unless `default`)
- ii. If multiple cases are ready: runtime picks pseudo-randomly (uniform over ready cases)
- iii. Stopping:
 - `time.After(d)`: channel that fires after `d`
 - `done` channel to signal stop

For-select Loop:

```
for {
    select {
        case <-done:
            return
        case <-time.After(1 * time.Second):
            // timed out
        default:
            // do work
    }
}
```

sync Package

Use mostly in small scopes (e.g., inside a `struct`); prefer higher-level coordination with channels when possible.

- i. WaitGroup: `var wg sync.WaitGroup`
 - `wg.Add(n)`, `wg.Done()`, `wg.Wait()`, `wg.Go(f)`
- ii. Mutex: `var mu sync.Mutex`
 - Locks: `mu.Lock()`, `mu.Unlock()`
- iii. Read-Write Mutex: `var rw sync.RWMutex`
 - Reads: `rw.RLock()`, `rw.RUnlock()`
 - Writes: `rw.Lock()`, `rw.Unlock()`
- iv. Cond: `c := sync.NewCond(&mu)`
 - Wait: `c.Wait()` unlocks & blocks, then re-locks before returning
 - Wake: `c.Signal()` / `c.Broadcast()`
- v. Once: `once.Do(f)` runs `f` at most once
- vi. Pool: `p.Get()` / `p.Put(x)`

Memory Model

Go's Memory Model:

- i. Sequenced-Before: goroutine program order
- ii. Synchronized-Before:
 - `go f()` $\rightarrow$ goroutine `f` begins
 - Channel send $\rightarrow$ matching receive completes
 - `close(c)` $\rightarrow$ zero-value receive from closed `c`
 - Unbuffered receive $\rightarrow$ matching send completes
 - Buffered k^{th} receive $\rightarrow (k + C)^{th}$ send completes
- iii. Happens-Before: transitive visibility relation of Sequenced-Before $\cup$ Synchronized-Before
- iv. Guarantee a read `r` of variable `v` observes write `w` if:
 - `w` happens before `r`
 - Any other write to `v` happens before `w` or after `r`

Concurrency Patterns

Patterns:

- i. Confinement:
 - Ad-hoc: only one goroutine mutates shared data
 - Lexical: restrict access to shared locations (e.g., expose only `<-chan T`, slice of array)
- ii. Prevent goroutine leaks:
 - Creator ensures termination when work done, unrecoverable error, or when told to stop working
 - Beware nil channels: read/write blocks forever
- iii. Error handling:
 - Return result + error along the same channel
 - Centralize decisions in a state-goroutine
- iv. Pipeline:
 - Stages connected by channels; each stage is a group of goroutines running the same function
- v. Fan-out / Fan-in:
 - Fan-out: replicate a slow stage across goroutines
 - Fan-in: multiplex input chans into 1 output chan
- vi. Load balancing:
 - Pool of workers reads from a shared work channel
 - Each worker emits results; collect with fan-in

Confinement

```
func owner() <-chan int {
    out := make(chan int) // owned by function

    go func() {
        defer close(out) // owner closes channel
        for i := 0; i < 5; i++ {
            out <- i // only owner goroutine sends
        }
    }()

    return out // expose receive-only channel
}
```

For-Select Loop

```
func worker(done <-chan struct{}, in <-chan int) {
    for {
        select {
        case <-done:
            return
        case x, ok := <-in:
            if !ok { return }
            work(x)
        }
    }
}
```

Prevent Goroutine Leak

```
func doWork(done <-chan struct{},
            in <-chan string) <-chan struct{} {
    terminated := make(chan struct{})

    go func() {
        defer close(terminated) // signal goroutine
        exited

        for {
            select {
            case s, ok := <-in:
                if !ok { return } // no blocked goroutine
                left behind
                fmt.Println(s)
            case <-done:
                return // caller can force termination
            }
        }
    }()

    return terminated // caller can wait for cleanup
}
```

Error Handling

```
type Result struct {
    Err error
    Resp *http.Response
}

func check(done <-chan struct{},
           urls ...string) <-chan Result {
    out := make(chan Result)

    go func() {
        defer close(out)

        for _, u := range urls {
            resp, err := http.Get(u)
            r := Result{Err: err, Resp: resp}

            select {
            case <-done:
                return
            case out <- r: // send value/error together
            }
        }
    }()

    return out
}
```

Pipeline

```
func gen(xs ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, x := range xs {
            out <- x
        }
    }()
    return out
}

func sq(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for x := range in {
            out <- x * x
        }
    }()
    return out
}

// stages compose: gen -> sq -> range
for x := range sq(gen(2, 3, 4)) {
    fmt.Println(x)
}
```

Fan-Out / Fan-In

```
func fanIn(cs ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup

    for _, c := range cs {
        wg.Add(1)
        go func(c <-chan int) {
            defer wg.Done()
            for x := range c { out <- x }
        }(c)
    }

    go func() { wg.Wait(); close(out) }()
    return out
}

// fan-out: many workers share randInts
for p := range fanIn(
    primeFinder(randInts),
    primeFinder(randInts),
    primeFinder(randInts),
) {
    fmt.Println(p)
}
```

Load Balancing

```
type Request struct {
    fn func() int
    reply chan int // per-request return path
}

func worker(reqs <-chan Request) {
    for r := range reqs { // workers pull load
        r.reply <- r.fn()
    }
}

func requester(reqs chan<- Request) {
    reply := make(chan int)

    for { // continuously generates load
        reqs <- Request{workFn, reply}
        result := <-reply // wait for own result
        process(result)
    }
}

reqs := make(chan Request) // shared work q
for i := 0; i < runtime.NumCPU(); i++ {
    go worker(reqs) // worker pool balances load
}

go requester(reqs)
```

4. Classical Synchronisation Problems

Producer-Consumer

Processes share a buffer, where producers add items and consumers remove items when not empty.

Finite Buffer (init spaces = K, items = 0):

```
// Producer      // Consumer
x = produce();   wait(items);
                wait(mutex);
wait(spaces);     x = buffer.get();
wait(mutex);     signal(mutex);
                buffer.add(x);
signal(mutex);    signal(spaces);
signal(items);    consume(item);
```

Reader-Writer

Processes share a critical region, where readers can read simultaneously but writers must have exclusive access.

```
// Reader
wait(mutex);
readers++;
if (readers == 1)
    wait(empty);
signal(mutex);

// Writer
wait(empty);
write();
signal(empty);

read();

wait(mutex);
readers--;
if (readers == 0)
    signal(empty);
signal(mutex);
```

- Writer Starvation: possible if readers keep entering and `empty` is never signalled. Solutions:
 - Queue: all pass through `wait(queue)`
 - Writer-Priority: readers entering must `wait(readTry)`; but first waiting writer blocks new readers with `wait(readTry)` and does `signal(readTry)` when done

Barrier

C++:

```
std::barrier b(N, on_done);

auto work = [&](string name) {
    cout << name << " worked\n";
    b.arrive_and_wait();

    cout << name << " cleaned\n";
    b.arrive_and_wait();
};
```

Go:

```
type Barrier struct {
    wg sync.WaitGroup
    wg2 sync.WaitGroup
}

func (b *Barrier) Init(n int) {
    b.wg.Add(n)
    b.wg2.Add(n)
}

func (b *Barrier) Wait() {
    b.wg.Done()
    b.wg.Wait()
    b.wg.Add(1)

    b.wg2.Done()
    b.wg2.Wait()
    b.wg2.Add(1)
}
```

Sleeping Barber

Barber sleeps until a customer wakes him. If all waiting chairs are full, the customer leaves.

C++:

(customer, barber, customerDone, barberDone = 0):

```
// Customer
{
    scoped_lock lk{mut};
    if (customers ==
        MaxCustomers + 1) {
        balk();
        return;
    }
    customers++;

// Barber
while (true) {
    customer.acquire();
    barber.release();

    cutHair();

    custDone.acquire();
    barbDone.release();
}
```

Go

(chairs : chan int [N], barber : chan struct {}):

```
// Barber
for {
    bs.barber <-
        struct{}{}
    <-bs.chairs
    cutHair()
}

// Customer
select {
case bs.chairs <- 1:
    <-bs.barber
    getHairCut()
default:
    balk()
}
```

Dining Philosophers

N philosophers around a circular table, with a single chopstick between. 2 chopsticks are needed to eat.

```
// Philosopher
void p(int i) {      // Take Chopstick
    while (1) {      void take(int i) {
        // think      wait(mutex);
        take(i);      state[i] =
        eat();         HUNGRY;
        put(i);        safe_to_eat(i);
    }                 signal(mutex);
                    wait(s[i]);
    }                 }

void safe_to_eat
    (int i) {        // Put Chopstick
    if (state[i] ==  void put(int i) {
        HUNGRY &&    wait(mutex);
        state[LEFT] != state[i] =
        EATING &&    THINKING;
        state[RIGHT] !=
        EATING) {    safe_to_eat(LEFT);
        state[i] =
        EATING;      safe_to_eat(RIGHT)
        signal(s[i]); signal(mutex);
    }                }
}
```

Limited Eater Solution, when at most $N - 1$ philosophers eat concurrently, it's guaranteed at least one can eat:

```
void philosopher(int i) {
    while (1) {
        // think
        wait(seats);
        wait(chopstick[LEFT]);
        wait(chopstick[RIGHT]);
        eat();
        signal(chopstick[RIGHT]);
        signal(chopstick[LEFT]);
        signal(seats);
    }
}
```

C++:

```
struct DiningTable {
    FairMutex chopstick[N];
    FairSemaphore footman{N - 1};

    FairMutex& left(size_t i) {
        return chopstick[i];
    }

    FairMutex& right(size_t i) {
        return chopstick[(i + 1) % N];
    }

    void eat(size_t i) {
        footman.acquire();
        {
            std::scoped_lock lk{left(i), right(i)};
            eat_cb(i);
        }
        footman.release();
    }
};
```

Go:

```
type ChpStick struct{}
type DiningTable struct {
    n int
    chs []chan ChpStick
}

func (t *DiningTable) Init(n int) {
    t.n = n
    for i := 0; i < n; i++ {
        ch := make(chan ChpStick, 1)
        ch <- ChpStick{}
        t.chs = append(t.chs, ch)
    }
}

// left, right defined same as C++
// odd-even order avoids circular wait
func (t *DiningTable) Eat(pid int) {
    first, second := t.left(pid), t.right(pid)
    if pid%2 == 1 {
        first, second = second, first
    }

    <-first
    <-second
    eat_cb(pid)
    second <- ChpStick{}
    first <- ChpStick{}
}
```

H2O

C++:

```
struct WaterFactory {
    std::counting_semaphore<> oxygenSem{1};
    std::counting_semaphore<> hydrogenSem{2};
    std::barrier<> bar{3};

    void oxygen(void (*bond)()) {
        oxygenSem.acquire(); // at most 1
        bar.arrive_and_wait(); // wait for HHO
        bond();
        oxygenSem.release();
    }

    void hydrogen(void (*bond)()) {
        hydrogenSem.acquire(); // at most 2
        bar.arrive_and_wait(); // wait for HHO
        bond();
        hydrogenSem.release();
    }
};
```

Go:

```
type WaterFactory struct {
    oxygenMutex chan struct{}
    precomH chan chan struct{}
}

// Init with oxygenMutex <- struct{}{}

func (wf *WaterFactory) Hydrogen(bond func()) {
    commit := make(chan struct{})

    wf.precomH <- commit // precommit
    <-commit              // commit
    bond()
    commit <- struct{}{} // postcommit
}

func (wf *WaterFactory) Oxygen(bond func()) {
    <-wf.oxygenMutex // become leader

    h1 := <-wf.precomH
    h2 := <-wf.precomH

    h1 <- struct{}{} // release 2 H atoms
    h2 <- struct{}{}
    bond()
    <-h1 // wait for 2 H atoms to finish
    <-h2

    wf.oxygenMutex <- struct{}{} // step down
}
```

5. Rust

Ownership Model

Rust enforces memory-safety at compile-time without GC, preventing data races and enabling concurrency.

- i. Each value has exactly one owner
- + Ownership prevents double-free; owner frees
- + Borrowing prevents use-after-free (UAF)
- + Segfault-free

Operations:

- i. Move: `T`
 - Transfers ownership; old binding cannot be used
 - Deep Copy: `clone()` is explicit (expensive)
- ii. Shared Borrow: `&T`
 - Both bindings are read-only while shared
- iii. Mutable Borrow: `&mut T`
 - Original binding cannot be used while borrowed
 - Borrowed binding has exclusive mutable access

Threads

Operations (`std::thread`):

- i. Spawn: `let h = thread::spawn(move || {...})`
 - `move`: moves captured values into thread
 - Captured values must be `Send + 'static`
- ii. Scoped Threads: `thread::scope(...)`
 - Threads finish before scope exits
 - Can borrow local non-`'static` data safely
- iii. Join: `h.join()`
- iv. Marker Traits:
 - `Copy`: safe to memcpy (i.e. `u32`, `f32`, no `String`)
 - `Send`: safe to transfer to another thread
 - `Sync`: safe to share reference between threads;
`T is Sync ⇔ &T is Send`
- v. Shared Ownership:
 - `Rc<T>`: within a thread
 - `Arc<T>`: across threads if `T` is thread-safe

Move vs Borrow

```
fn take(v: Vec<i32>) { } // takes ownership
fn read(v: &Vec<i32>) { } // shared borrow
```

```
let v = vec![1, 2, 3];
read(&v); // v still usable
take(v); // v moved
// read(&v); // compile error
```

Spawn + Join

```
let v = vec![1, 2, 3];

// move transfers v into the spawned thread
let h = thread::spawn(move || {
    println!("{:?}", v);
    42 // thread return value
});

// println!("{:?}", v); // compile error: moved
let result = h.join().unwrap();
println!("{}", result);
```

Shared State Across Threads

```
let c = Arc::new(Mutex::new(0));
let mut hs = vec![];

for _ in 0..4 {
    let c = Arc::clone(&c);
    hs.push(thread::spawn(move || {
        *c.lock().unwrap() += 1;
    }));
}

for h in hs { h.join().unwrap(); }

println!("{}", *c.lock().unwrap());
```

Scoped Threads

```
let mut xs = vec![1, 2, 3, 4];

thread::scope(|s| {
    let (a, b) = xs.split_at_mut(2);

    s.spawn(|| for x in a { *x += 1 });
    s.spawn(|| for x in b { *x += 1 });
}); // all scoped threads joined here

println!("{:?}", xs);
```

Mutexes

Operations (`std::sync::Mutex`):

- i. Construct: `let m = Mutex::new(x)`
- ii. Lock-Guard: `let mut g = m.lock().unwrap()`
 - Guard controls access: `*g += 1`
 - When guard goes out of scope, mutex unlocks
- iii. Poisoning:
 - Panic while holding lock makes later `lock()` return `Err`
 - `m.lock().unwrap()` propagates poisoning as panic
- iv. Interior Mutability:
 - `Mutex<T>` allows mutation through shared ownership with synchronization

Atomics

Atomics provide indivisible shared-memory operations.

Operations (`std::sync::atomic`):

- i. Construct: `let a = AtomicUsize::new(v)`
 - `AtomicBool`, `AtomicIsize`, `AtomicI16`
- ii. Load: `a.load(SeqCst)`
- iii. Store: `a.store(v, SeqCst)`
- iv. Read-Modify-Write:
`a.fetch_add(1, SeqCst)`, `a.swap(v, SeqCst)`

Multi-Producer, Single Consumer (MPSC)

MPSC provides a FIFO communication queue.

Operations (`std::sync::mpsc`):

- i. Create: `let (tx, rx) = channel()`
- ii. Clone Sender: `let tx2 = tx.clone()`
- iii. Send: `tx.send(v)`
- iv. Receive: `rx.recv()`

crossbeam Package

Crossbeam extends beyond `std`.

Features:

- i. Bounded Channels: `crossbeam::bounded(C)`
- ii. Backoff: `crossbeam_utils::Backoff::new()`
 - Exponentially slow retry reduces contention
 - Busywait: `.spin()` / Yield: `.snooze()`

rayon Package

Rayon provides data-parallelism operations.

Features (`rayon::prelude::*`):

- i. Iterators: `xs.par_iter()`, `xs.par_iter_mut()`
- ii. Combinators:
 - `map(...)`, `filter(...)`, `for_each(...)`
 - Aggregation: `reduce(...)` / `sum()`
- iii. Sort: `xs.par_sort()` / `xs.par_sort_unstable()`

Parallel Max

```
let max = AtomicI64::new(MIN);
vec.par_iter().for_each(|n| {
    loop {
        let old = max.load(SeqCst);
        if *n <= old { break; }
        let r = max.compare_and_swap(old, *n, SeqCst);
        if r == old {
            // swapped
            break;
        }
    }
})
```

Non-blocking I/O

Non-blocking I/O enables concurrency with one thread:

- i. Blocking I/O: thread sleeps until data arrives
- ii. `epoll`: kernel reports ready file descriptors
- iii. Event Loop: wait for ready fds → handle I/O

Futures

Futures represent work that will produce a value later:

- i. `Future`: trait driven by `poll(Context)`:
 - Runs until it can no longer make progress
 - Returns `Poll::Ready(T)` or `Poll::Pending`
- ii. `Context` provides `wake()` (notify executor to repoll)
- iii. Combinators: `.then()`

Executors are user-space schedulers that drive futures:

- i. `Pending` ⇒ future arranges a `wake()` when it can progress again
- ii. If nothing can progress: sleep until a `wake()`

Async/Await

Async/Await are syntactic sugar for composing Futures.

- i. `async fn` returns a `Future`
 - ii. `.await` waits for a future (only inside `async`)
- + Zero-cost Abstraction: compiler lowers `async fn` into a state machine implementing `poll()`
 - + Lower memory use with stackless coroutines
 - + Lower scheduling overhead from context switches
 - + Scales to very high concurrency (esp. I/O bound)
 - Block inside `async` sleeps the executor thread
 - Cooperative: tasks yield only at `.await`
 - CPU-heavy work can starve other tasks
 - No recursion; future size fixed at compile time

Tokio

Tokio is an async runtime: executor + reactor + async I/O.

Features:

- i. Runtime macro: `#[tokio::main]`
- ii. Spawn: `tokio::spawn(async move {...})`
- iii. Join: `handle.await`
- iv. Blocking work:
`tokio::task::spawn_blocking(...)`
- v. Async sync:
`tokio::sync::{Mutex, Semaphore, Barrier}`

Tokio Tasks

```
#[tokio::main]
async fn main() {
    let mut hs = vec![];

    for i in 0..4 {
        hs.push(tokio::spawn(async move {
            work(i).await
        }));
    }

    for h in hs {
        println!("{:?}", h.await.unwrap());
    }
}
```

6. Testing

Bug Types:

- i. Unwanted Blocking:
 - Deadlock/Livelock: no progress, operations are not "completed", effects do not become visible
 - IO: waiting for external input
- ii. Race Conditions:
 - Data races $\Rightarrow$ undefined behavior (unsynchronized concurrent access)
 - Broken invariants:
 - Double-free
 - Dangling pointers $\Rightarrow$ UAF
 - Memory corruption from read partial update
 - Lifetime issues: thread outlives data it accesses

Techniques:

- i. Review the code:
 - Identify shared data and how it is protected
 - Track held mutexes and find lock ordering issues
 - Check cross-thread ordering enforcement
 - Validate lifetimes (e.g. `join()` on exceptions)
- ii. Testing:
 - Run the smallest amount of code that can demonstrate the problem
 - Remove concurrency to confirm it is concurrency-related
 - Run on multicore and (when useful) single-core

Dynamic Instrumentation

Valgrind:

- i. Shadows registers and memory to validate memory operations:
each allocation freed once, each access to allocated space, each reach from initialized memory
- ii. High overhead: 10-20x
- iii. Usage: `valgrind --tool=memcheck <prog ...>`

Helgrind:

- i. Intercepts `pthread` operations to validate thread operations:
misuse of POSIX threads API, potential deadlocks from lock-order cycles, data races from happens-before analysis
- ii. Very high overhead: ~100x
- iii. Usage: `valgrind --tool=helgrind <prog ...>`

Compiler Instrumentation

Sanitizers:

- i. Compile-time instrumentation; supported by `clang` and `gcc`
- ii. Lower overhead: ~5-10x
- iii. Instruments memory accesses / function boundaries and tracks memory state

AddressSanitizer (ASan):

- i. Detects addressability issues
- ii. Typical cost: ~2x slowdown, 1.5-3x memory overhead
- iii. Usage: `-fsanitize=address`

ThreadSanitizer (TSan):

- i. Detects data races using happens-before tracking
- ii. Also reports thread / mutex / lock-order issues
- iii. Typical cost: 5-15x slowdown, 5-10x memory overhead
- iv. Usage: `-fsanitize=thread -fPIE -pie`

Model Checking

Check Formal Specifications:

- i. Build a model in a special DSL / new language
 - ii. Check: constraints, unexpected behaviour, deadlock
 - iii. Why spend time:
 - Check designs before costly implementation
 - Prove properties for existing code
 - Enable aggressive optimizations without breaking correctness
- + Rigorous, check all traces exhaustively
+ Produces trace violating requirement
– Tedious to define correct specification

Approaches:

- i. Write a formal specification and check it (with model checker/ proof assistant)
- ii. Use specification to write minimal code:
- iii. Add invariants as comments to assist checker:

Model Checkers:

- i. TLA+ (TLC): temporal properties; good for concurrent/distributed systems
 - Write a TLA+ specification; use TLC to exhaustively explore the state space
 - Check safety (invariants) and liveness (temporal)
 - Reports errors with a counterexample trace
 - Model Structure:
`Init` (initial states) + `Next` (state transitions)
 - Uses fairness to rule out infinite "stuttering" (non-progress) executions
- ii. Coq: interactive proofs; generate code (OCaml/Haskell/Scheme)
- iii. Alloy: relational logic; good for modeling structures

Challenges of Distributed Systems:

- i. Reliability: in case of failures
- ii. No global clock or ordering: need memory model
- iii. Consistency
- iv. Consensus